

P1815R2: Translation-unit-local entities

Audience: CWG

S. Davis Herring <herring@lanl.gov>

Los Alamos National Laboratory

February 14, 2020

History

r2:

- Rebased onto N4849
- Added changes specified by P2003R1
 - Allowed non-exposure references and local references to entities internal to header units
 - Allowed multiple realizations of a header unit
 - Prohibited in header units entities that cannot be defined in multiple translation units
- Added changes specified by P1884R0
- Allowed exposures in private module fragments
- Excluded use of constant-initialized references as an exposure
- Prohibited redundant implicit instantiations of inline variables
- Combined explicit instantiation wording and adjusted it for modules and member classes
- Defined TU-local for values for exposures via constant expressions
- Made unnamed types without an associated key be TU-local
- Corrected requirements for definitions of inline functions/variables and templates
- Covered missing kinds of “naming”
- Simplified naming to be independent of translation units
- Clarified (uninstantiated) inline function template case
- Removed leftover deduction guide wording
- Fixed overbroad [dcl.inline] note
- Added motivation for TU-locality of classes without linkage
- Fixed paragraph markers

r1:

- Added example and fixed existing example
- Added NB comment reference
- Moved deduction guide wording to correct place
- Restored basic permission for `static` in header units
- Made more template specializations TU-local
- Clarified type specifiers
- Clarified TU-locality and usage of unnamed types

Introduction

This paper is a replacement for [P1498R1](#) that includes the wording promised for it and thereby addresses [US35](#). While its general principle of forbidding exposure of internal-linkage constructs to other translation units is followed, this paper presents rules that differ for consistency, especially for templates. It also includes wording (which would conflict) for [P2003R1](#) and [P1884R0](#) and thereby addresses [US133](#) and [US134](#) (and, in a sense, [US36](#)).

The function to which a dependent name refers cannot, of course, be determined merely from the template declaration that contains it. [P1347R1](#) proposed forbidding such a call that selects an internal-linkage function via overload resolution. Such overload resolution can in general require undesirably extensive analysis of otherwise-local declarations, so [P1498R1](#) goes further and (with a single exception for certain constants) forbids any use of such a name in a context where code generation might occur for another translation unit. (This includes, for example, the return type of a non-template function with more than internal linkage; while it need not be mangled, it is accessible via `decltype` anywhere the function is.) We can consider such overload sets to be *poisoned* by their mere inclusion of an unusable candidate. The unusable candidate might be a template; templates whose declarations involve internal entities must themselves be internal to avoid needing to perform template argument deduction for them (even if the resulting specialization might not involve any internal types). (As a reminder, `export` is irrelevant here because the client translation unit might be in the same module.)

However, ADL considers a set of functions and function templates which itself cannot be known until instantiation. It is therefore impossible to reliably preclude poisoned overload sets by any reasonable restriction on (non-internal) templates.

This paper therefore merely proscribes the external use of internal entities and of poisoned overload sets, and does so for the definition of a template only during instantiation. Even a template that unambiguously calls an internal function can have non-internal linkage so long as the only specializations used in other translation units are those generated by an explicit specialization or instantiation. (Implicit instantiation of a template does not preclude other implicit instantiations and so does not allow usage from other translation units.) Local/anonymous classes (including closure types) are internal based on where they are defined, regardless of whether they (attempt to) use internal entities, partly to avoid any need to include in an invented name for the class an internal-linkage name introduced by the enclosing definition.

It would of course be possible to discard certain overload resolution candidates before deciding whether an overload set is poisoned. Most simply, any that do not support the number of arguments given in the call could be excluded. More sophisticated analysis might exclude functions or function templates with parameter types not convertible from a (non-dependent) argument type, as well as those whose constraints are known not to be met for any instantiation. Preferring future flexibility and compatibility over invention, this paper simply poisons overload sets unconditionally.

Wording

Relative to N4849.

Change [basic.def.odr]/10:

- Every program shall contain exactly one definition of every non-inline function or variable that is odr-used in that program outside of a discarded statement ([stmt.if]); no diagnostic required. The definition can appear explicitly in the program, it can be found in the standard or a user-defined library, or (when appropriate) it is implicitly defined (see [class.default.ctor], [class.copy.ctor], [class.dtor], and [class.copy.assign]). ~~A definition of an inline function or variable shall be reachable in every translation unit in which it is odr-used outside of a discarded statement.~~ [Example:

[...]

— end example]

Insert before [basic.def.odr]/11:

- A *definition domain* is a *private-module-fragment* or the portion of a translation unit before its *private-module-fragment* (if any). A definition of an inline function or variable shall be reachable from the end of every definition domain in which it is odr-used outside of a discarded statement.

Change [basic.lookup.argdep]/5:

- [Example:

Translation unit #1:

```
export module M;
namespace R {
    export struct X {};
    export void f(X);
}
namespace S {
    export void f(R::X, R::X);
}
```

Translation unit #2:

```
export module N;
import M;
export R::X make();
namespace R { static int g(X); }
export template<typename T, typename U> void apply(T t, U u) {
    f(t, u);
    g(t);
}
```

Translation unit #3:

```
module Q;
import N;
namespace S {
    struct Z { template operator T(); };
}
void test() {
    auto x = make();           // OK, decltype(x) is R::X in module M
    R::f(x);                   // error: R and R::f are not visible here
    f(x);                      // OK, calls R::f from interface of M
    f(x, S::Z());              // error: S::f in module M not considered
                                // even though S is an associated namespace
    apply(x, S::Z());          // OK, error: S::f is visible in instantiation context, and but
                                // R::g is visible even though it has internal linkage and cannot be used outside TU #2
}
```

— end example]

Append paragraphs to [basic.link]:

- A declaration *D* names an entity *E* if
 1. *D* contains a *lambda-expression* whose closure type is *E*,

2. *E* is not a function or function template and *D* contains an *id-expression*, *type-specifier*, *nested-name-specifier*, *template-name*, or *concept-name* denoting *E*, or
3. *E* is a function or function template and *D* contains an expression that names *E* ([basic.def.odr]) or an *id-expression* that refers to a set of overloads that contains *E*. [Note: Non-dependent names in an instantiated declaration do not refer to a set of overloads ([temp.nondep]). — end note]

- A declaration is an *exposure* if it either names a TU-local entity (defined below), ignoring

1. the *function-body* for a non-inline function or function template (but not the deduced return type for a (possibly instantiated) definition of a function with a declared return type that uses a placeholder type ([dcl.spec.auto])),
2. the *initializer* for a variable or variable template (but not the variable's type),
3. friend declarations in a class definition, and
4. any reference to a non-volatile const object or reference with internal or no linkage initialized with a constant expression that is not an odr-use ([basic.def.odr]),

or defines a constexpr variable initialized to a TU-local value (defined below). [Note: An inline function template can be an exposure even though explicit specializations of it might be usable in other translation units. — end note]

- An entity is *TU-local* if it is

1. a type, function, variable, or template that
 1. has a name with internal linkage, or
 2. does not have a name with linkage and is declared, or introduced by a *lambda-expression*, within the definition of a TU-local entity,
2. a type with no name that is defined outside a *class-specifier*, function body, or *initializer* or is introduced by a *defining-type-specifier* that is used to declare only TU-local entities,
3. a specialization of a TU-local template,
4. a specialization of a template with any TU-local template argument, or
5. a specialization of a template whose (possibly instantiated) declaration is an exposure. [Note: The specialization might have been implicitly or explicitly instantiated. — end note]

- A value or object is TU-local if either

1. it is, or is a pointer to, a TU-local function or the object associated with a TU-local variable,
2. it is an object of class or array type and any of its subobjects or any of the objects or functions to which its non-static data members of reference type refer is TU-local and is usable in constant expressions.

- If a (possibly instantiated) declaration of, or a deduction guide for, a non-TU-local entity in a module interface unit (outside the *private-module-fragment*, if any) or module partition ([module.unit]) is an exposure, the program is ill-formed. Such a declaration in any other context is deprecated ([depr.local]).

- If a declaration that appears in one translation unit names a TU-local entity declared in another translation unit that is not a header unit, the program is ill-formed. A declaration instantiated for a template specialization ([temp.spec]) appears at the point of instantiation of the specialization ([temp.point]).

- [Example:

Translation unit #1:

```
export module A;
static void f() {}
inline void it() {f();} // error: is an exposure of f
static inline void its() {f();} // OK
template<int> void g() {its();} // OK
template void g<0>();

decltype(f) *fp; // error: f (though not its type) is TU-local
auto &fr=f; // OK
constexpr auto &fr2=fr; // error: is an exposure of f
constexpr static auto fp2=fr; // OK

struct S {void (&ref)();} s{f}; // OK: value is TU-local
constexpr extern struct W {S &s;} wrap{s}; // OK: value is not TU-local

static auto x=[]{f();} // OK
auto x2=x; // error: the closure type is TU-local
int y=[]{f();}(),0; // error: the closure type is not TU-local
int y2=(x,0); // OK

namespace N {
    struct A {};
    void adl(A);
    static void adl(int);
}
void adl(double);

inline void h(auto x) {adl(x);} // OK, but a specialization might be an exposure
```

Translation unit #2:

```
module A;
void other() {
    g<0>(); // OK: specialization is explicitly instantiated
    g<1>(); // error: instantiation uses TU-local its
    h(N::A{}); // error: overload set contains TU-local N::adl(int)
    h(0); // OK: calls adl(double)
    adl(N::A{}); // OK: N::adl(int) not found; calls N::adl(N::A)
    fr(); // OK: calls f
    constexpr auto ptr=fr; // error: fr is not usable in constant expressions here
}
```

— end example]

Insert before [expr.const]/3:

- A variable is *potentially-constant* if it is constexpr or it has reference or const-qualified integral or enumeration type.

Change [expr.const]/3:

- A *constant-initialized potentially-constant* variable is *usable in constant expressions* after at a point *P* if its initializing declaration *D* is encountered if reachable from *P* and
 1. it is a constexpr variable, or it is a constant-initialized variable of reference type or of const-qualified integral or enumeration type
 2. it is not initialized to a TU-local value, or
 3. *P* is in the same translation unit as *D*.

An object or reference is *usable in constant expressions* if it is

[...]

Change [dcl.inline]/2:

- A function declaration ([dcl.fct], [class.mfct], [class.friend]) with an inline specifier declares an *inline function*. The inline specifier indicates to the implementation that inline substitution of the function body at the point of call is to be preferred to the usual function call mechanism. An implementation is not required to perform this inline substitution at the point of call; however, even if this inline substitution is omitted, the other rules for inline functions specified in this subclause shall still be respected. [Note: The inline keyword has no effect on the linkage of a function. In certain cases, an inline function cannot use names with internal linkage; see [basic.link]. — end note]

Change and split [dcl.inline]/6:

- If an inline function or variable is odr-used in a translation unit, a definition of it shall be reachable from the end of that translation unit, and it shall have exactly the same definition in every such translation unit ([basic.def.odr]). [Note: A call to the inline function or a use of the inline variable may be encountered before its definition appears in the translation unit. — end note] If a definition of a function or variable is reachable at the point of its first declaration as inline, the program is ill-formed. If a function or variable with external or module linkage is declared inline in one translation unit definition domain, there shall be a reachable inline declaration in all translation units of it shall be reachable from the end of every definition domain in which it is declared; no diagnostic is required. [Note: A call to an inline function or a use of an inline variable may be encountered before its definition becomes reachable in a translation unit. — end note]
- [Note: An inline function or variable with external or module linkage shall have has the same address in all translation units. [Note: A static local variable in an inline function with external or module linkage always refers to the same object. A type defined within the body of an inline function with external or module linkage is the same type in every translation unit. — end note]

Change [dcl.inline]/7:

- An exported inline function or variable that is attached to a named module is declared in a definition domain, it shall be defined in the translation unit containing its exported declaration, outside the private-module-fragment (if any) that domain. [Note: There is no restriction on the linkage (or absence thereof) of entities that the function body of an exported inline function can reference. A constexpr function ([dcl.constexpr]) is implicitly inline. — end note]

Change [dcl.spec.auto]/10:

- An exported function with a declared return type that uses a placeholder type shall be defined in the translation unit containing its exported declaration, outside the private-module-fragment (if any). [Note: There is no restriction on the linkage of the The deduced return type cannot have a name with internal linkage ([basic.link]). — end note]

Change and split [module.import]/5:

- A module-import-declaration that specifies a header-name *H* imports a synthesized header unit, which is a translation unit formed by applying phases 1 to 7 of translation ([lex.phases]) to the source file or header nominated by *H*, which shall not contain a module-declaration. [Note: All declarations within a header unit are implicitly exported ([module.interface]), and are attached to the global module ([module.unit]). — end note] An importable header is a member of an implementation-defined set of headers that includes all importable C++ library headers ([headers]). *H* shall identify an importable header. Two Given two such module-import-declarations;

1. ~~import the same header unit if and only if their header-names identify the same~~ different headers or source files ([cpp.include]), ~~they import distinct header units;~~
2. ~~otherwise, if they appear in the same translation unit, they import the same header unit;~~
3. ~~otherwise, it is unspecified whether they import the same header unit. [Note: It is therefore possible that multiple copies exist of entities declared with internal linkage in an importable header. — end note]~~

[Note: A module-import-declaration nominating a header-name is also recognized by the preprocessor, and results in macros defined at the end of phase 4 of translation of the header unit being made visible as described in [cpp.import]. — end note]

- A declaration of a name with internal linkage is permitted within a header unit despite all declarations being implicitly exported ([module.interface]). ~~If such a declaration declares an entity that is odr-used outside the header unit, or by a template instantiation whose point of instantiation is outside the header unit, the program is ill-formed. [Note: A definition that appears in multiple translation units cannot in general refer to such names ([basic.def.odr]). — end note] A header unit shall not contain a definition of a non-inline function or variable whose name has external linkage.~~

Change [module.private.frag]/2:

- [Note: A private-module-fragment ends the portion of the module interface unit that can affect the behavior of other translation units. A private-module-fragment allows a module to be represented as a single translation unit without making all of the contents of the module reachable to importers. The presence of a private-module-fragment affects:
 1. the point by which the definition of an exported inline function is required ([dcl.inline]),
 2. the point by which the definition of an exported function with a placeholder return type is required ([dcl.spec.auto]),
 3. ~~whether a declaration is required not to be an exposure ([basic.link]).~~
 4. ~~where definitions for inline functions and templates must appear ([basic.def.odr], [temp.pre]).~~
 5. the instantiation contexts of templates instantiated before it ([module.context]), and
 6. the reachability of declarations within it ([module.reach]). — end note]

Change [temp.pre]/10:

- A ~~definition of a~~ function template, member function of a class template, variable template, or static data member of a class template shall be ~~defined in~~ ~~reachable from the end of~~ every translation unit definition domain ([basic.def.odr]) in which it is implicitly instantiated ([temp.inst]) unless the corresponding specialization is explicitly instantiated ([temp.explicit]) in some translation unit; no diagnostic is required.

Change [temp.dep.constexpr]/2.5:

- it ~~is a constant with literal type and~~ ~~names a potentially-constant variable ([expr.const]) that~~ is initialized with an expression that is value-dependent.

Insert before [temp.inst]/1:

- A template specialization *E* is a *declared specialization* if there is a reachable explicit instantiation definition ([temp.explicit]) or explicit specialization declaration ([temp.expl.spec]) for *E*, or if there is a reachable explicit instantiation declaration for *E* and *E* is not
 1. an inline function,
 2. declared with a type deduced from its initializer or return value ([dcl.spec.auto]),
 3. a potentially-constant variable ([expr.const]), or
 4. a specialization of a templated class.

[Note: An implicit instantiation in an importing translation unit cannot use names with internal linkage from an imported translation unit ([basic.link]). — end note]

Change [temp.inst]/1:

- Unless a class template specialization ~~has been explicitly instantiated ([temp.explicit]) or explicitly specialized ([temp.expl.spec]) is a~~ ~~declared specialization~~, the class template specialization is implicitly instantiated when the specialization is referenced in a context that requires a completely-defined object type or when the completeness of the class type affects the semantics of the program. [...]

Change [temp.inst]/3:

- Unless a member of a class template or a member template ~~has been explicitly instantiated or explicitly specialized is a~~ ~~declared specialization~~, the specialization of the member is implicitly instantiated when the specialization is referenced in a context that requires the member definition to exist or if the existence of the definition of the member affects the semantics of the program; in particular, the initialization (and any associated side effects) of a static data member does not occur unless the static data member is itself used in a way that requires the definition of the static data member to exist.

Change [temp.inst]/4:

- Unless a function template specialization ~~has been explicitly instantiated or explicitly specialized is a~~ ~~declared specialization~~, the function template specialization is implicitly instantiated when the specialization is referenced in a context that requires a function definition to exist or if the existence of the definition affects the semantics of the program. A function whose declaration was instantiated from a friend function definition is implicitly instantiated when it is referenced in a context that requires a function definition to exist or if the existence of the definition affects the semantics of the program. Unless a call is to a function template explicit specialization or to a member function of an explicitly specialized class template, a default argument for a function template or a member function of a class template is implicitly

instantiated when the function is called in a context that requires the value of the default argument. *[Note: An inline function that is the subject of an explicit instantiation declaration is not a declared specialization; the intent is that it still be implicitly instantiated when odr-used ([basic.def.odr]) so that the body can be considered for inlining, but that no out-of-line copy of it be generated in the translation unit. — end note]*

Change [temp.inst]/6:

- Unless a variable template specialization ~~has been explicitly instantiated or explicitly specialized~~ *is a declared specialization*, the variable template specialization is implicitly instantiated when it is referenced in a context that requires a variable definition to exist or if the existence of the definition affects the semantics of the program. A default template argument for a variable template is implicitly instantiated when the variable template is referenced in a context that requires the value of the default argument.

Remove [temp.explicit]/14:

- Except for inline functions [...]

Add a subclass before [depr.impldec]:

- Non-local use of TU-local entities [depr.local]

A declaration of a non-TU-local entity that is an exposure ([basic.link]) is deprecated. *[Note: Such a declaration in an importable module unit is ill-formed. — end note]* *[Example:*

```
namespace {
    struct A {
        void f() {}
    };
}
A h();
inline void g() {A().f();} // deprecated: inline and not internal linkage
```

— end example]